

# Atividade 15 | Autorização com Roles, JWT e Prisma

Implementação de controle de acesso declarativo baseado em perfis no NestJS

## Estudante

Milena Hamerski

## Disciplina

Tópicos Especiais: Desenvolvimento Web Avançado

## Instituição

UTFPR - Universidade Tecnológica Federal do Paraná

## Data de Execução

04 de Junho de 2026

## Perguntas para Reflexão

### 1. O que é o decorator `@Roles()` e o que ele faz nos bastidores?

O decorator `@Roles()` é um decorator de metadados personalizado criado utilizando a função auxiliar `SetMetadata()` fornecida pelo NestJS. Nos bastidores, ele associa uma lista de papéis (roles) definidos (como `'ADMIN'` ou `'USER'`) a um manipulador de rota (método de controller) ou à classe do controller como um todo. Esses metadados são indexados sob uma chave única específica (`'roles'`) e podem ser lidos dinamicamente em tempo de execução por guards (como o `RolesGuard`) ou interceptores por meio da classe utilitária `Reflector` do NestJS para determinar se a execução deve prosseguir.

### 2. Qual a função do `RolesGuard`? Por que ele é importante e por que deve vir sempre depois do `JwtAuthGuard`?

O `RolesGuard` é um guard de autorização encarregado de avaliar se o usuário que realiza a requisição possui os privilégios de acesso necessários requeridos pela rota. Ele lê as roles associadas ao endpoint via `@Roles()` e as compara com as permissões atribuídas ao usuário ativo.

Ele deve vir sempre **após** o `JwtAuthGuard` porque o processo de autorização depende estritamente de um usuário já autenticado. O `JwtAuthGuard` extrai, valida o token JWT do cabeçalho da requisição e anexa o payload decodificado (o objeto contendo id, email e roles) no objeto `request.user`. Se o `RolesGuard` fosse executado antes, ele tentaria ler `request.user.roles` de um objeto nulo ou indefinido, gerando um erro de referência ou resultando no bloqueio incorreto de qualquer acesso.

### 3. O que acontece se uma rota tiver `@Roles('ADMIN')` mas o `RolesGuard` não for aplicado no Controller?

Caso a rota possua a anotação `@Roles('ADMIN')`, mas o `RolesGuard` não seja adicionado via `@UseGuards(RolesGuard)` (seja no escopo do método, do controller ou de forma global), a lógica de controle de acesso não será executada. O decorator `@Roles()` apenas anexa metadados à rota, mas não possui nenhuma inteligência ativa para bloquear a execução. Sem o `RolesGuard` para ler esses metadados e validar o usuário, a rota ficará exposta para qualquer usuário autenticado (se o `JwtAuthGuard` estiver ativo) ou até mesmo não autenticado (se nenhum guard estiver ativo).

## Resultados dos Testes de Integração (Passo 8 & 10)

Os testes a seguir foram realizados utilizando requisições programáticas automatizadas em um banco de dados real PostgreSQL gerenciado pelo Neon. As requisições mostram o fluxo de registro, atribuição de cargos no banco com o Prisma, autenticação com JWT e a validação do **bypass para Administradores** (Desafio Extra).

### 1. Criação e Login do Usuário Comum (Role USER)

Registro e login de um usuário comum que recebe o papel USER por padrão:

#### **POST /auth/register**

Response: {"id":"28c5ab93-072d-470a-883a-7cc832e9e3d7","name":"Test User","email":"test\_user\_activity15@example.com","age":25,"roles":["USER"]}

#### **POST /auth/login**

Response: {"access\_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyOGM1Y..."}

---

## 2. Criação, Promoção e Login do Administrador (Role ADMIN)

Registro de um segundo usuário e promoção do seu papel para ADMIN no banco de dados via Prisma Client, seguido do login:

### POST /auth/register

Response: {"id":"b585f14e-7a4e-4ac1-bdc3-bd7119e8b4d0","name":"Test Admin","email":"test\_admin\_activity15@example.com","age":30,"roles":["USER"]}

### Promote User to ADMIN in DB

Result: Successfully updated role for user: test\_admin\_activity15@example.com. Current roles: ["ADMIN"]

### POST /auth/login

Response: {"access\_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJiNTg1Z..."} }

## 3. Testes na Rota Protegida de Administração (GET /admin)

A rota exige o papel ADMIN. Veja os resultados com diferentes tipos de acesso:

Acesso sem token:

```
{"message": "Unauthorized", "statusCode": 401}
```

**HTTP Status: 401 Unauthorized**

Acesso com token de USER (Acesso Negado):

```
{"message": "Forbidden resource", "error": "Forbidden", "statusCode": 403}
```

**HTTP Status: 403 Forbidden**

Acesso com token de ADMIN (Acesso Permitido):

```
{"message": "Bem-vindo, Admin!"}
```

**HTTP Status: 200 OK**

## 4. Desafio Extra: Rota Exclusiva de Usuário com Bypass de ADMIN (GET /admin/somente-usuario)

A rota exige papel USER. Com o bypass implementado no RolesGuard, administradores possuem livre acesso a qualquer rota:

Usuário normal acessando a rota de USER:

```
{"message":"Você tem acesso como USER (ou ADMIN via bypass)!", "user": {"id":"28c5ab93-072d-470a-883a-7cc832e9e3d7", "name":"Test User", "roles":["USER"]}}
```

HTTP Status: 200 OK

Administrador acessando a rota de USER (Sucesso via Bypass):

```
{"message":"Você tem acesso como USER (ou ADMIN via bypass)!", "user": {"id":"b585f14e-7a4e-4ac1-bdc3-bd7119e8b4d0", "name":"Test Admin", "roles":["ADMIN"]}}
```

HTTP Status: 200 OK